

TRƯỜNG CÔNG NGHỆ THÔNG TIN & TRUYỀN THÔNG - ĐẠI HỌC BÁCH KHOA HÀ NỘI

BÁO CÁO ĐỒ ÁN PROJECT II

HỆ THỐNG GIÁM SÁT VÀ TỐI ƯU HÓA LOGISTICS THEO THỜI GIAN THỰC

Giảng viên hướng dẫn: TS. Nguyễn Hồng Phương
Sinh viên thực hiện: Thân Văn Long Nhật
Mã sinh viên: 20215626
Mã học phần: IT3943
Số điện thoại: 0946253718
Lớp: Việt Nhật K66 - Hệ đào tạo: Kỹ sư chất lượng cao
Khóa học (K): K66
Thời gian thực hiện: Học kỳ 2025.2 (Từ tháng 03/2026 đến tháng 06/2026)

Hà Nội, 06/2026

MỤC LỤC

- CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI VÀ NHIỆM VỤ CỦA ĐỒ ÁN
 - 1.1. Giới thiệu đề tài
 - 1.2. Phát biểu bài toán và vấn đề
 - 1.3. Mục tiêu của đồ án
 - 1.4. Phạm vi và giới hạn của đồ án
 - 1.5. Nhiệm vụ cụ thể của đồ án
- CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG
 - 2.1. Cơ sở lý thuyết
 - 2.1.1. Bài toán định tuyến xe động (DVRP)

- [2.1.2. Giải thuật di truyền \(Genetic Algorithm - GA\)](#)
- [2.1.3. Thuật toán Dijkstra trên đồ thị giao thông động](#)
- [2.1.4. Xử lý dữ liệu luồng vị trí và Map-matching](#)
- [2.2. Công nghệ sử dụng](#)
 - [2.2.1. Apache Kafka](#)
 - [2.2.2. Apache Spark Structured Streaming](#)
 - [2.2.3. Redis](#)
 - [2.2.4. MongoDB và Change Streams](#)
 - [2.2.5. Node.js và Socket.IO](#)
 - [2.2.6. React.js và Leaflet Canvas Rendering](#)
 - [2.2.7. Docker và Kubernetes](#)

[3. CHƯƠNG 3. PHÂN TÍCH YÊU CẦU VÀ THIẾT KẾ XÂY DỰNG HỆ THỐNG](#)

- [3.1. Phân tích yêu cầu chức năng](#)
- [3.2. Thiết kế dữ liệu và cấu trúc bản tin \(Schema\)](#)
 - [3.2.1. Luồng dữ liệu GPS thô \(Kafka Topic\)](#)
 - [3.2.2. Bản đồ mạng lưới đường \(JSON Schema\)](#)
 - [3.2.3. Trạng thái giao thông thời gian thực \(Redis Key\)](#)
 - [3.2.4. Lộ trình và đơn hàng giao \(MongoDB Document\)](#)
- [3.3. Thiết kế kiến trúc hệ thống](#)
 - [3.3.1. Sơ đồ kiến trúc 5 tầng](#)
 - [3.3.2. Luồng dữ liệu và giao tiếp thời gian thực](#)
 - [3.3.3. Thiết kế giao thức WebSockets \(Socket.IO events\)](#)
 - [3.3.4. Mô hình triển khai microservices trên Kubernetes](#)
- [3.4. Thiết kế chi tiết thuật toán và logic nghiệp vụ](#)
 - [3.4.1. Tầng Stream Processing \(Spark Structured Streaming\)](#)
 - [3.4.2. Tầng Route Optimization Worker](#)
 - [3.4.3. Cơ chế xử lý điểm cắt \(Sink Nodes\) bằng](#)

[giải thuật Reorder Sinks Last](#)

- [3.4.4. Tương tác Dashboard & Cập nhật lạc quan \(Optimistic UI Updates\)](#)

4. [CHƯƠNG 4. TRIỂN KHAI THỰC NGHIỆM VÀ KẾT QUẢ](#)

- [4.1. Kịch bản và môi trường thử nghiệm](#41-kịch-bản-và-môi-trường-thử-nghiệm)
- [4.2. Hướng dẫn chi tiết chụp ảnh kết quả thực nghiệm](#)
- [4.3. Đánh giá kết quả đạt được](#)
- [4.4. Bài học kinh nghiệm và hạn chế](#)
- [4.5. Hướng phát triển tương lai](#)
- [4.6. Kết luận chung](#)

5. [TÀI LIỆU THAM CHIẾU](#)

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI VÀ NHIỆM VỤ CỦA ĐỒ ÁN

1.1. Giới thiệu đề tài

Trong các hệ thống phân phối hàng hóa và logistics đô thị hiện nay, việc định tuyến hành trình cho phương tiện (Vehicle Routing) đóng vai trò quyết định đến chi phí vận hành và thời gian đáp ứng khách hàng. Tuy nhiên, tại các thành phố lớn có mật độ giao thông phức tạp như Hà Nội, tình hình giao thông luôn biến động không ngừng theo thời gian thực do ùn tắc vào giờ cao điểm, thời tiết xấu hoặc các rào cản vật lý phát sinh đột xuất. Nếu chỉ áp dụng các phương pháp tìm đường tĩnh tại thời điểm xuất phát, các phương tiện dễ dàng rơi vào khu vực tắc nghẽn, dẫn đến trễ hẹn giao hàng và làm lãng phí nhiên liệu của đội xe.

Do đó, nhu cầu cấp thiết đặt ra là xây dựng một hệ thống có khả năng giám sát trực quan vị trí di chuyển của toàn bộ đội xe và tình trạng ùn tắc giao thông trên từng tuyến phố theo thời gian thực. Đồng thời, hệ thống cần hỗ trợ người điều phối tương tác gán đơn hàng mới trực tiếp từ bản đồ và tự động tính toán lại lộ trình di chuyển tối ưu nhất cho xe để tránh các khu vực tắc nghẽn. Đồ án **Project II** này tập trung nghiên cứu thiết kế và triển khai

một **Hệ thống giám sát và tối ưu hóa logistics theo thời gian thực**, ứng dụng kiến trúc hướng sự kiện (Event-driven Microservices) kết hợp giữa các thuật toán tối ưu hóa tổ hợp (Giải thuật di truyền GA, Dijkstra động) và các công nghệ truyền thông, lưu trữ tốc độ cao.

1.2. Phát biểu bài toán và vấn đề

Bài toán chính mà đề án giải quyết là **Bài toán định tuyến xe động phụ thuộc thời gian thực (Dynamic Vehicle Routing Problem - DVRP)** trên bản đồ giao thông số thuộc khu vực quận Đống Đa và Ba Đình, thành phố Hà Nội.

Hệ thống cần quản lý một tập hợp các phương tiện vận chuyển (xe tải) xuất phát từ các vị trí ngẫu nhiên trên bản đồ. Nhiệm vụ của mỗi xe tải là di chuyển qua một tập hợp các địa chỉ khách hàng được người điều phối yêu cầu giao hàng để thực hiện trả đơn. Các vấn đề cốt lõi cần giải quyết bao gồm:

1. **Tiếp nhận luồng tọa độ GPS thô cực lớn:** Module mô phỏng phương tiện liên tục phát tọa độ GPS của đội xe với chu kỳ 1 giây/lần. Hệ thống phải tiếp nhận thông suốt luồng dữ liệu này mà không xảy ra nghẽn mạng hay mất mát bản tin.

2. **Khớp bản đồ (Map-matching) gần thời gian thực:** Tọa độ GPS thô do có sai số vị trí nên cần được ánh xạ chính xác vào danh sách các cạnh đường (đoạn phố) tương ứng trên đồ thị bản đồ.

3. **Phát hiện và cảnh báo tắc nghẽn tự động:** Hệ thống phải liên tục gom nhóm dữ liệu GPS trong các cửa sổ thời gian gần nhất để tính vận tốc trung bình của dòng giao thông trên từng cạnh đường, từ đó phát hiện các đoạn đường bị ùn ứ (vận tốc trung bình ≤ 10 km/h) và cập nhật tức thời vào kho dữ liệu chia sẻ.

4. **Tái định tuyến động (Dynamic Rerouting):** Khi xe đang di chuyển theo lộ trình cũ nhưng có cạnh đường phía trước đột ngột bị tắc nghẽn, hoặc khi người điều phối tạo thêm các đơn hàng giao mới cho xe, hệ thống phải tự động tính toán lại thứ tự giao hàng tối ưu và tìm tuyến đường cạnh liên mạch mới để tránh điểm ùn tắc.

5. **Giao tiếp hai chiều và tương tác trực quan:** Người dùng trên giao diện Dashboard có thể theo dõi xe di chuyển mượt mà, đổi màu

các cạnh đường theo mức độ tắc nghẽn và thực hiện chọn xe, click trực tiếp lên bản đồ để giao đơn hàng mới.

1.3. Mục tiêu của đồ án

Mục tiêu tổng quát của đồ án là thiết kế và xây dựng thành công một hệ thống phần mềm hoàn chỉnh theo mô hình microservices, tích hợp các thuật toán tối ưu lộ trình thông minh và giao tiếp thời gian thực.

Các mục tiêu cụ thể bao gồm:

- Thiết kế và cài đặt module mô phỏng (Bot Simulator) có khả năng sinh luồng dữ liệu GPS giả lập cho tối đa 100 xe tải giao hàng và 5.000 phương tiện nền di chuyển trên bản đồ.
- Thiết lập hạ tầng truyền thông điệp trung gian Apache Kafka để xử lý luồng dữ liệu vị trí với độ tin cậy cao.
- Xây dựng chương trình xử lý luồng Spark Structured Streaming để map-matching vị trí xe và tính toán mật độ giao thông động theo cửa sổ thời gian (sliding window).
- Áp dụng Redis để duy trì trạng thái tắc nghẽn tức thời và MongoDB để lưu trữ dữ liệu lộ trình bền vững.
- Thiết kế và cải tiến giải thuật tối ưu hóa lộ trình kết hợp giữa Giải thuật di truyền (GA) để sắp xếp chuỗi giao hàng và thuật toán Dijkstra động để tìm đường tránh tắc nghẽn, đồng thời xử lý triệt để các góc cụt (sink nodes) trên đồ thị đường.
- Xây dựng Dashboard tương tác thời gian thực bằng React và Node.js kết hợp WebSockets, hỗ trợ chế độ giao đơn hàng trực tiếp bằng chuột và hiển thị tiến độ giao hàng của từng xe.
- Đóng gói toàn bộ hệ thống bằng Docker và triển khai ổn định trên môi trường Kubernetes nội bộ.

1.4. Phạm vi và giới hạn của đồ án

- **Phạm vi dữ liệu bản đồ:** Hệ thống tập trung thử nghiệm trên mạng lưới đường phố khu vực quận Đống Đa và Ba Đình, Hà Nội. Bản đồ đường được trích xuất từ OpenStreetMap (OSM) và chuyển đổi thành đồ thị kề dạng JSON gồm khoảng hơn 1.000 nút và cạnh đường.
- **Mô phỏng dữ liệu:** Dữ liệu GPS và tình trạng tắc nghẽn được tạo ra thông qua các bot giả lập trong chương trình chạy nền để đảm bảo tính chủ động trong kịch bản kiểm thử, chưa kết nối trực tiếp với thiết bị GPS phần cứng thực tế hoặc API của các nhà cung cấp bản đồ thương mại (như Google Maps).
- **Quy mô triển khai:** Hệ thống được đóng gói để triển khai cục bộ (local deployment) trên một máy tính cá nhân sử dụng Kubernetes ảo (Minikube). Do đó, các tài nguyên CPU, RAM được tối ưu hóa để toàn bộ các microservices và hạ tầng cơ sở dữ liệu có thể chạy đồng thời mà không làm sập hệ thống.

1.5. Nhiệm vụ cụ thể của đồ án

Để hoàn thành mục tiêu đề ra, sinh viên cần thực hiện các nhiệm vụ kỹ thuật cụ thể sau:

1. **Tìm hiểu lý thuyết** về bài toán DVRP, giải thuật di truyền (GA), thuật toán Dijkstra và các mô hình thiết kế hệ thống microservices hướng sự kiện.
2. **Thiết kế cấu trúc dữ liệu** thống nhất cho toàn bộ các thành phần của hệ thống bao gồm: schema bản ghi GPS, schema đồ thị bản đồ, schema lưu trữ traffic state trong Redis và schema lộ trình xe trong MongoDB.
3. **Phát triển module Ingestion:** Cài đặt Kafka Broker và Zookeeper trên Kubernetes, lập trình bot simulator gửi tọa độ GPS vào Kafka topic `gps_stream`.
4. **Phát triển module Stream Processing:** Viết ứng dụng Spark Structured Streaming bằng Python đọc dữ liệu từ Kafka, thực hiện khớp vị trí Euclidean, tính tốc độ trung bình và đẩy trạng thái vào Redis.

5. **Phát triển module Route Optimization:** Viết worker chạy nền giám sát các thay đổi lộ trình, cài đặt Genetic Algorithm tối ưu hóa TSP cho các điểm giao hàng và Dijkstra để nối các cạnh đường tránh kẹt xe. Xây dựng logic xử lý nút cụt (sink nodes) bằng cách BFS tìm số nút liên thông.

6. **Phát triển Web Dashboard:** Viết Express Backend hỗ trợ kết nối MongoDB Change Streams và Socket.IO server. Viết React Frontend vẽ bản đồ Leaflet Canvas, hỗ trợ chọn xe, bắt sự kiện click chuột để đặt điểm giao hàng, hiển thị tiến độ đơn hàng và cập nhật giao diện ngay lập tức khi người dùng click (Optimistic Updates).

7. **Triển khai và kiểm thử hệ thống:** Viết các file cấu hình Kubernetes YAML cho toàn bộ hạ tầng và các microservices, thực hiện deploy, viết script seed dữ liệu và kiểm chứng tính năng tự động tái định tuyến khi hệ thống phát hiện tắc đường.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

2.1. Cơ sở lý thuyết

2.1.1. Bài toán định tuyến xe động (DVRP)

Bài toán định tuyến phương tiện (Vehicle Routing Problem - VRP) là bài toán tối ưu hóa tổ hợp tìm kiếm tập hợp các lộ trình tốt nhất cho một đội xe phục vụ một danh sách khách hàng phân tán. Trong môi trường thực tế, bài toán được phát triển thành **Bài toán định tuyến phương tiện động (Dynamic VRP - DVRP)**, trong đó các thông số của bài toán có thể thay đổi hoặc xuất hiện mới trong quá trình xe đang di chuyển.

Trong đề án này, tính chất "động" của bài toán được mô hình hóa qua hai khía cạnh:

1. **Thời gian di chuyển phụ thuộc thời gian:** Thời gian di chuyển ước lượng qua một cạnh đường e là một hàm phụ thuộc vào vận tốc trung bình thực tế tại thời điểm di chuyển t : $T(e, t)$

$= \frac{L(e)}{V_{\text{realtime}}(e)}$ Với $L(e)$ là chiều dài vật lý của cạnh đường và $V_{\text{realtime}}(e)$ là vận tốc trung bình của dòng giao thông được cập nhật thời gian thực vào Redis từ luồng dữ liệu Spark.

2. Sự xuất hiện của các đơn hàng mới (On-demand Orders):

Trong lúc xe đang thực hiện giao hàng, người điều phối có thể thêm ngẫu nhiên hoặc chỉ định các điểm giao hàng mới trực tiếp trên bản đồ. Khi đó, danh sách khách hàng cần giao của xe thay đổi đột ngột, buộc hệ thống phải kích hoạt tính toán lại lộ trình để chèn điểm giao hàng mới một cách hợp lý nhất.

2.1.2. Giải thuật di truyền (Genetic Algorithm - GA)

Giải thuật di truyền là một thuật toán Heuristic tìm kiếm tối ưu hóa toàn cục dựa trên quy luật chọn lọc tự nhiên. Đối với bài toán tìm thứ tự giao hàng đi qua nhiều khách hàng (tương tự bài toán Người bán hàng - TSP), khi số lượng khách hàng tăng lên, không gian nghiệm tăng theo cấp số nhân ($n!$), khiến thuật toán duyệt cạn không thể đáp ứng thời gian thực thi ngắn. GA được sử dụng để nhanh chóng tìm ra một thứ tự giao hàng tối ưu.

Mô hình GA trong đồ án được thiết kế cụ thể như sau:

- **Mã hóa cá thể:** Sử dụng hoán vị các định danh khách hàng (Permutation Encoding). Một cá thể đại diện cho một thứ tự giao hàng cụ thể, ví dụ: $C = [\text{Cust}_1, \text{Cust}_3, \text{Cust}_2, \text{Cust}_4]$.
- **Hàm thích nghi (Fitness Function):** GA cố gắng tìm kiếm cá thể có tổng chi phí di chuyển là nhỏ nhất. Hàm fitness cho cá thể C được tính toán bằng cách tổng hợp thời gian di chuyển ước lượng giữa các điểm giao liên tiếp:
$$\text{Fitness}(C) = \sum_{i=1}^{n-1} \text{EstimateTravelTime}(K_i, K_{i+1}) + P_{\text{congestion}}$$
 Trong đó, $\text{EstimateTravelTime}$ giữa hai khách hàng được ước lượng dựa trên khoảng cách địa lý Haversine và vận tốc trung bình của khu vực. $P_{\text{congestion}}$ là điểm phạt nếu tuyến đường hiện tại đi qua đoạn đường bị tắc nghẽn (blocked edges).

- **Phép chọn lọc (Selection):** Sử dụng kết hợp **Rank Selection** (chọn các cá thể có thứ hạng cao về độ thích nghi làm cha mẹ) và **Elitism** (giữ lại 2 cá thể xuất sắc nhất từ thế hệ cha sang thế hệ con mà không qua đột biến, giúp bảo toàn lời giải tốt nhất qua các vòng lặp).
- **Phép lai (Crossover):** Áp dụng phép lai thứ tự (**Order Crossover - OX**). Phép lai này chọn một đoạn con ngẫu nhiên từ cá thể cha 1 sao chép sang con, các vị trí còn lại được điền bằng các phần tử của cha 2 theo thứ tự xuất hiện của chúng nhằm đảm bảo con sinh ra là một hoán vị hợp lệ của danh sách khách hàng mà không bị trùng lặp hay thiếu điểm giao.
- **Phép đột biến (Mutation):** Áp dụng đột biến hoán đổi (**Swap Mutation**). Chọn ngẫu nhiên hai khách hàng trong chuỗi hành trình và đổi chỗ cho nhau với xác suất mutation rate bằng 0.15, giúp thuật toán mở rộng không gian tìm kiếm nghiệm mới.

2.1.3. Thuật toán Dijkstra trên đồ thị giao thông động

Khi đã tìm được thứ tự các khách hàng cần giao bằng GA, hệ thống cần tìm ra một tuyến đường chi tiết (danh sách các cạnh đường kề nhau liên tục) để phương tiện di chuyển trên bản đồ. Thuật toán Dijkstra được sử dụng để tìm đường đi ngắn nhất giữa điểm xuất phát hiện tại của xe tới khách hàng thứ nhất, và tiếp tục nối từ khách hàng thứ i sang khách hàng thứ $i+1$.

Trọng số của mỗi cạnh e trên đồ thị giao thông được điều chỉnh động theo trạng thái giao thông đọc từ Redis: $W(e) = \begin{cases} \frac{L(e)}{V_{\text{realtime}}(e)} & \text{nếu } e \notin \text{blocked_edges} \\ \infty & \text{(hoặc phạt } 100 \text{ lần)} & \text{nếu } e \in \text{blocked_edges} \end{cases}$ Trọng số động giúp thuật toán Dijkstra tự động hướng tuyến đường đi vòng qua các phố thông thoáng khác thay vì đi xuyên qua phố đang bị kẹt xe nặng, ngay cả khi quãng đường vật lý có thể dài hơn.

2.1.4. Xử lý dữ liệu luồng vị trí và Map-matching

Hệ thống tiếp nhận dòng sự kiện vị trí GPS thô gồm tọa độ vĩ độ, kinh

độ (lat, lon) được gửi liên tục theo thời gian thực.

- **Xử lý luồng:** Dữ liệu GPS được xử lý theo dạng micro-batch với tần suất vài giây một lần. Việc xử lý tức thời này giúp cập nhật kịp thời trạng thái giao thông ngắn hạn mà không cần chờ đợi lưu trữ toàn bộ dữ liệu vào đĩa cứng.

- **Map-matching:** Áp dụng phương pháp khớp điểm gần nhất (Euclidean Distance Nearest-Neighbor matching). Với mỗi tọa độ GPS nhận được $(\text{lat}_p, \text{lon}_p)$, hệ thống tính khoảng cách vuông góc từ điểm này tới tất cả các cạnh đường lân cận trong bán kính quét 30 mét: $\text{dist}(p, e) = \min_{t \in [0, 1]} \text{Distance}(p, (1-t) \cdot \text{start_node}(e) + t \cdot \text{end_node}(e))$ Cạnh đường e có khoảng cách ngắn nhất sẽ được gán là vị trí thực tế của xe tải.

2.2. Công nghệ sử dụng

2.2.1. Apache Kafka

Apache Kafka đóng vai trò là hệ thống hàng đợi thông điệp (Message Broker) phân tán hướng sự kiện. Với cơ chế ghi đĩa tuần tự và phân vùng dữ liệu, Kafka tiếp nhận luồng GPS từ Bot Simulator gửi vào topic `gps_stream` với thông lượng lớn và độ trễ cực thấp. Nó đóng vai trò là lớp đệm tách biệt (decoupling) giữa module sinh dữ liệu và module tiêu thụ (Spark, Backend), giúp hệ thống hoạt động ổn định kể cả khi tải tăng đột biến.

2.2.2. Apache Spark Structured Streaming

Apache Spark Structured Streaming là framework xử lý luồng dữ liệu phân tán mạnh mẽ. Spark đọc dữ liệu từ Kafka dưới dạng một bảng vô tận (Unbounded Table), thực hiện giải nén dữ liệu JSON, khớp tọa độ vào đồ thị bản đồ và chạy cửa sổ thời gian trượt (Sliding Window) để liên tục tính toán vận tốc trung bình của các xe chạy qua từng cạnh đường trong khoảng thời gian gần nhất.

2.2.3. Redis

Redis là cơ sở dữ liệu lưu trữ trong bộ nhớ (In-memory Database) với tốc độ truy xuất siêu nhanh (dưới 1ms). Redis được sử dụng để lưu trữ trạng thái giao thông động dưới dạng key-value và quản lý tập hợp `blocked_edges`. Cơ chế đặt thời gian sống (TTL - Time to Live) được áp dụng để tự động dọn dẹp các trạng thái giao thông đã quá cũ khi không có phương tiện nào di chuyển qua đoạn đường đó sau 2 phút, giúp giải phóng bộ nhớ.

2.2.4. MongoDB và Change Streams

MongoDB là cơ sở dữ liệu NoSQL lưu trữ dưới dạng tài liệu (Document-oriented). MongoDB lưu trữ thông tin lâu dài về lộ trình, danh sách các đơn hàng và trạng thái hoàn thành đơn hàng của các xe tải.

Bằng cách cấu hình MongoDB dưới dạng một cụm ReplicaSet, hệ thống kích hoạt tính năng **Change Streams**. Tính năng này cho phép Node.js Backend lắng nghe trực tiếp mọi thay đổi trên cơ sở dữ liệu (ví dụ khi Route Optimization Worker ghi đè một lộ trình mới tối ưu vào MongoDB) để phát sóng sự kiện cập nhật tức thời tới React Frontend mà không cần chạy cơ chế hỏi tuần hoàn (polling) gây quá tải đĩa.

2.2.5. Node.js và Socket.IO

- **Node.js:** Nền tảng thực thi JavaScript phía Server hoạt động theo mô hình hướng sự kiện và non-blocking I/O, rất phù hợp cho các ứng dụng thời gian thực có nhiều kết nối đồng thời.
- **Socket.IO:** Thư viện hỗ trợ giao tiếp hai chiều dựa trên giao thức WebSocket giữa Dashboard Backend và Frontend. Socket.IO quản lý các kết nối thời gian thực, tiếp nhận sự kiện gán đơn hàng từ người dùng và phát sóng thông tin cập nhật vị trí xe, trạng thái tắc đường tới trình duyệt.

2.2.6. React.js và Leaflet Canvas Rendering

- **React.js:** Thư viện giao diện người dùng dựa trên component, giúp xây dựng trang quản trị hiển thị mượt mà.

- **Leaflet Canvas Rendering:** Bản đồ Leaflet thông thường vẽ các marker và đường đi dưới dạng phần tử SVG trong cây DOM của trình duyệt. Khi số lượng xe và cạnh đường tăng lên hàng ngàn, trình duyệt sẽ bị quá tải DOM dẫn tới giật lag giao diện. Bằng cách cấu hình Leaflet sử dụng Canvas Rendering API, toàn bộ xe cộ và đường đi được vẽ trực tiếp trên một lớp Canvas phẳng duy nhất, giúp Dashboard có hiệu năng cực cao, duy trì tốc độ hiển thị 60 FPS ổn định.

2.2.7. Docker và Kubernetes

- **Docker:** Đóng gói mã nguồn và toàn bộ môi trường chạy của từng microservice (Node.js, React, Spark, Python GA Worker) thành các Docker Image độc lập, loại bỏ lỗi cấu hình không đồng nhất môi trường.
- **Kubernetes (Minikube):** Nền tảng điều phối container quản lý toàn bộ các thành phần hệ thống. Kubernetes chịu trách nhiệm quản lý vòng đời các Pod, ánh xạ mạng nội bộ thông qua Service DNS và tự động phục hồi (self-healing) khởi động lại các container bị lỗi, đảm bảo hệ thống vận hành liên tục 24/7.

CHƯƠNG 3. PHÂN TÍCH YÊU CẦU VÀ THIẾT KẾ XÂY DỰNG HỆ THỐNG

3.1. Phân tích yêu cầu chức năng

Hệ thống hướng tới việc đáp ứng đầy đủ các yêu cầu chức năng của một trung tâm điều phối vận tải logistics thông minh:

1. **Giám sát trực quan:** Bản đồ hiển thị vị trí thực tế của xe tải và các bot mô phỏng di chuyển theo thời gian thực. Xe tải đang di chuyển có badge trạng thái "Đang chạy" hoặc "Đang dừng".
2. **Bản đồ giao thông động:** Các đoạn đường trên bản đồ tự động đổi màu dựa trên tốc độ dòng xe (Xanh: thông thoáng, Cam: di chuyển chậm, Đỏ: tắc nghẽn nặng).

3. **Quản lý đơn hàng tương tác (Assign Mode):** Cho phép người dùng nhấp chọn một xe tải cụ thể và click trực tiếp trên bản đồ để thêm các điểm giao hàng (đơn hàng). Giao diện Sidebar hiển thị danh sách đơn hàng đã gán và trạng thái tương ứng của từng đơn (pending - chờ tối ưu/chờ giao, next - đang trên đường tới, delivered - đã giao thành công).

4. **Tự động tối ưu hóa lộ trình:** Khi phát hiện đường phía trước bị tắc nghẽn hoặc khi có đơn hàng mới được thêm vào, hệ thống chạy thuật toán GA để sắp xếp lại chuỗi giao hàng và Dijkstra để định tuyến lại, giúp xe đi vòng tránh các đoạn đường đỏ.

5. **Reset/Clear đơn hàng:** Hỗ trợ người dùng xóa nhanh toàn bộ đơn hàng của một xe để gán lại từ đầu khi cần thiết.

3.2. Thiết kế dữ liệu và cấu trúc bản tin (Schema)

3.2.1. Luồng dữ liệu GPS thô (Kafka Topic)

Dữ liệu GPS thô do simulator phát vào Kafka topic `gps_stream` ở định dạng JSON để Spark và Backend tiêu thụ:

```
{
  "entity_id": "Truck_001",
  "entity_type": "Truck",
  "latitude": 21.012345,
  "longitude": 105.812345,
  "speed": 24.5,
  "edge_id": "E_105_TaySon",
  "timestamp": 1779176030000,
  "route_index": 5,
  "route_total": 35
}
```

Ý nghĩa các trường:

- `entity_id`: Tên định danh của phương tiện.
- `entity_type`: Loại phương tiện (Truck hoặc Bot).
- `latitude, longitude`: Tọa độ GPS thô.
- `speed`: Tốc độ di chuyển thực tế (km/h).
- `edge_id`: Cạnh đường hiện tại xe đang di chuyển.
- `timestamp`: Thời điểm sinh sự kiện (epoch ms).

3.2.2. Bản đồ mạng lưới đường (JSON Schema)

Mạng lưới đường khu vực thử nghiệm được định nghĩa trong file `edges_schema.json` làm cơ sở dữ liệu đồ thị chung cho Dijkstra và Map-matching:

```
{
  "edge_id": "E_105_TaySon",
  "road_name": "Tay Son",
  "max_speed_kmh": 40.0,
  "start_node": {
    "node_id": "N_101",
    "lat": 21.012000,
    "lon": 105.812000
  },
  "end_node": {
    "node_id": "N_102",
    "lat": 21.012800,
    "lon": 105.813000
  },
  "length_meters": 142.5
}
```

3.2.3. Trạng thái giao thông thời gian thực (Redis Key)

Redis lưu trữ trạng thái giao thông của từng cạnh đường tại key `edge:<edge_id>` với cấu trúc JSON:

```
{
  "edge_id": "E_105_TaySon",
  "avg_speed": 8.5,
  "vehicle_count": 12,
  "distance": 142.5,
  "max_speed": 40.0,
  "estimated_travel_time": 60.35,
  "is_congested": true,
  "updated_at": "2026-07-10T19:21:00",
  "last_updated": 1779176030000
}
```

Khi `avg_speed` ≤ 10 km/h, trường `is_congested` nhận giá trị `true` và cạnh đó được thêm vào Redis Set `blocked_edges`.

3.2.4. Lộ trình và đơn hàng giao (MongoDB Document)

MongoDB quản lý thông tin gán tuyến và danh sách đơn hàng của các xe trong collection `assigned_routes`:

```
{
  "_id": {"$oid": "664c3db523c4a20b08fa1234"},
  "vehicle_id": "Truck_001",
  "assigned_route": ["E_104_ChuaBoc", "E_105_TaySon",
    "E_106_NguyenLuongBang"],
  "new_assigned_route": ["E_104_ChuaBoc", "E_107_XaDan",
    "E_106_NguyenLuongBang"],
}
```

```
"customers": [  
  {  
    "cust_id": "Cust_Truck_001_171600000",  
    "latitude": 21.015600,  
    "longitude": 105.818000,  
    "order": 1,  
    "status": "next"  
  },  
  {  
    "cust_id": "Cust_Truck_001_171600005",  
    "latitude": 21.020500,  
    "longitude": 105.822000,  
    "order": 2,  
    "status": "pending"  
  }  
],  
"remaining_customers": [  
  {  
    "cust_id": "Cust_Truck_001_171600000",  
    "latitude": 21.015600,  
    "longitude": 105.818000  
  },  
  {  
    "cust_id": "Cust_Truck_001_171600005",  
    "latitude": 21.020500,
```

```

        "longitude": 105.822000

    }

],

"current_edge_id": "E_104_ChuaBoc",

"current_edge_index": 0,

"total_edges": 3,

"estimated_total_travel_time": 185.20,

"needs_optimization": false,

"rerouted": true,

"reroute_reason": "Tắc nghẽn tại Tay Son",

"last_optimized_at": 1779176040000,

"route_status": "optimized"

}

```

3.3. Thiết kế kiến trúc hệ thống

3.3.1. Sơ đồ kiến trúc 5 tầng

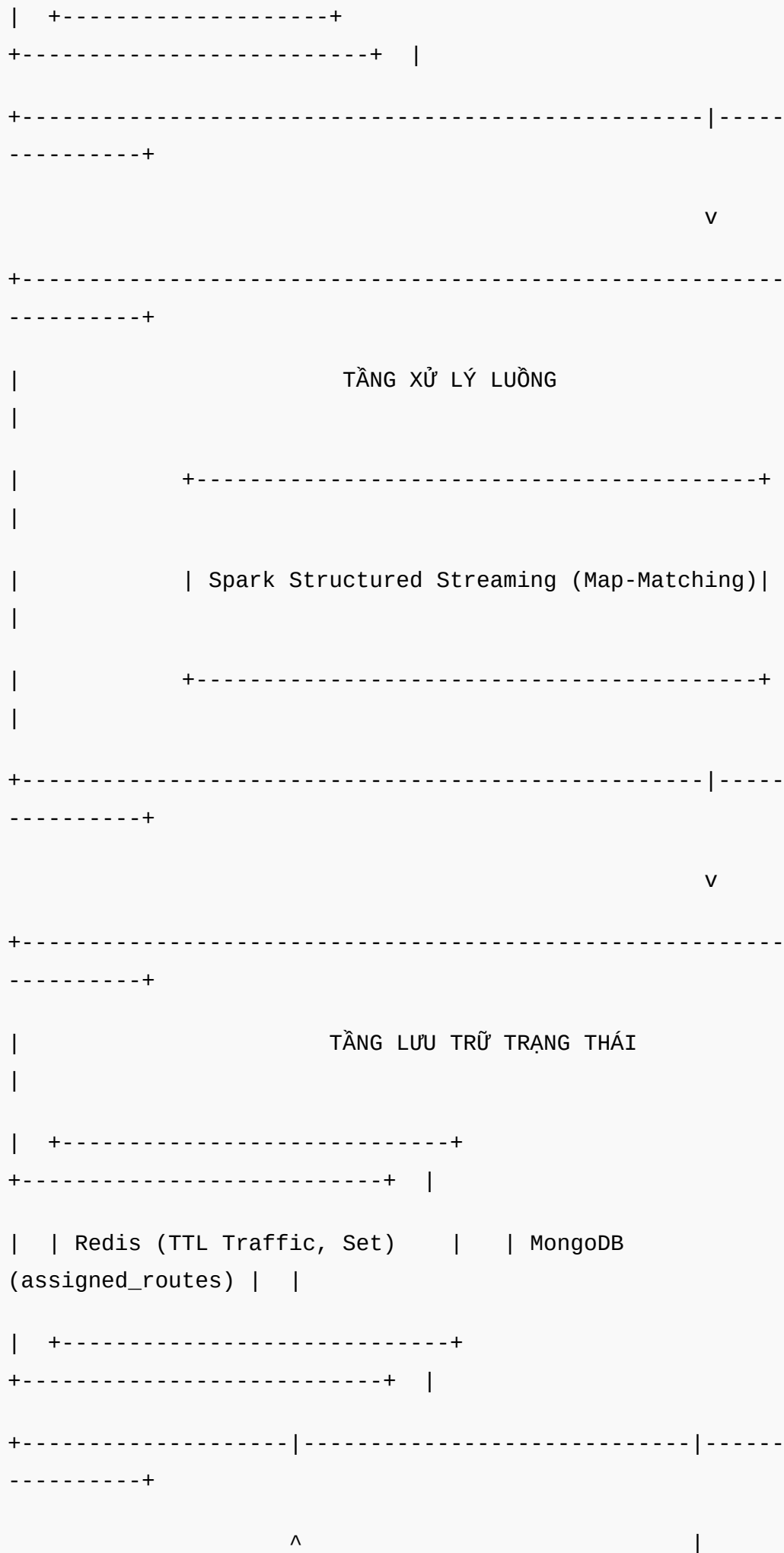
Kiến trúc hệ thống được phân rã thành 5 lớp dịch vụ độc lập nhằm đảm bảo khả năng bảo trì, mở rộng và tăng độ bền vững:

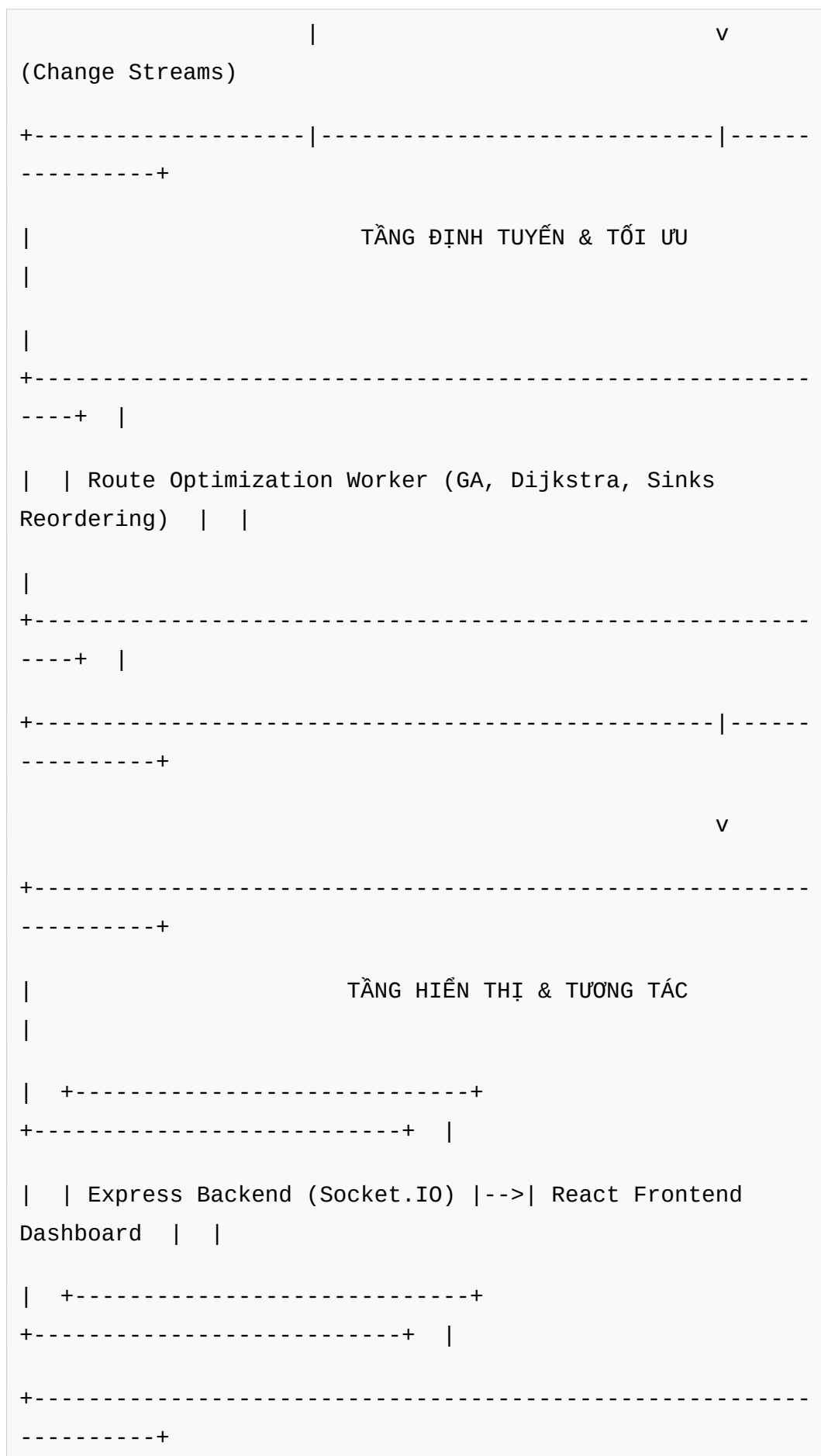
```

+-----+
|
|                                     TẦNG HẠ TẦNG & TRUYỀN SỰ KIỆN
|
|
|  +-----+
+-----+ |

|  |  Bot Simulator  |  --(GPS)-->  |  Kafka topic:
gps_stream  |  |

```





3.3.2. Luồng dữ liệu và giao tiếp thời gian thực

Hệ thống vận hành theo một vòng lặp kín hướng sự kiện (Closed-

Loop Event-Driven Data Flow):

1. Phương tiện di chuyển liên tục cập nhật GPS thô gửi tới **Kafka**.
2. **Spark Streaming** đọc theo micro-batch 5 giây, map-matching lên đồ thị và ghi đè trạng thái tốc độ cạnh vào **Redis**.
3. Người dùng click gắn đơn hàng cho xe trên bản đồ, **Frontend** phát lệnh tạo đơn thông qua **Socket.IO** tới **Backend Express**.
4. **Backend** nhận lệnh, cập nhật mảng customers và gắn cờ `needs_optimization = true` vào tài liệu của xe trong **MongoDB**.
5. **Route Optimizer Worker** định kỳ quét MongoDB mỗi 5 giây, phát hiện xe có cờ `needs_optimization = true` hoặc lộ trình hiện tại có giao với các cạnh bị kẹt xe trong Redis.
6. Worker chạy **GA** tìm thứ tự giao tối ưu, chạy **Dijkstra** (với trọng số cạnh là thời gian di chuyển thực tế từ Redis) nối các điểm thành tuyến cạnh liên tục, tự động đẩy các điểm cắt (**Sink Nodes**) về cuối lộ trình. Kết quả được lưu đè lại vào MongoDB.
7. **MongoDB Change Streams** bắt sự kiện sửa đổi lộ trình của xe, thông báo cho **Backend Express** ngay lập tức.
8. **Backend Express** phát sự kiện `route_optimized` qua WebSockets xuống **Frontend React** để vẽ lại đường đi tối ưu và cập nhật danh sách đơn hàng cho người dùng.

3.3.3. Thiết kế giao thức WebSockets (Socket.IO events)

Bảng thiết kế chi tiết các sự kiện truyền nhận WebSockets giữa Backend và Frontend:

Tên sự kiện	Hướng	Dữ liệu truyền tải
vehicle_batch	Backend	Ảnh vị trí xe

ô
tả
ng
hiệ
p
vụ

ấy
vị

Tên sự kiện

Hướng

Dữ liệu truyền tải

ô tả năng suất

trí

xe

để

fro

nte

nd

ren

der

ma

rke

r

(ch

u

kỳ

1s)

.

ấy

trạ

ng

thá

i

tốc

độ

để

fro

nte

nd

đổi

trạng thái
raffiac_batch

Bảng
ackend
\$\n
rightarr
ow\$
Fronten
d

Mảng trạng thái giao thông
gồm [{ edge_id, avg_speed }]

T
ên sự
kiện

H
ướng

D
ữ liệu
truyền
tải

ô
tả
ng
hiệ
p
vụ

mà
u
đư
ờn
g
(ch
u
kỳ
2s)
.

ử
dữ
liệ
u

r
outes
_snap
shot

B
ackend
\$\n
rightarr
ow\$
Fronten
d

M
ảng toàn
bộ lộ
trình
hiện có
của các
xe

lộ
trì
nh
khi
fro
nte
nd
kết
nối
lần
đầ
u.

r
eques
t_rou

F
rontend

{
vehicl
e_id,

ử

T
ên sự
kiện

H
ướng

D
ữ liệu
truyền
tải

te

\$\
rightarr
ow\$
Backen
d

lat,
lon }

c
reate
_orde
r

F
rontend
\$\
rightarr
ow\$
Backen
d

{
vehicl
e_id,
lat,
lon }

ô
tả
ng
hiệ
p
vụ

yê
u
cầ
u
tín
h
rou
te
nh
an
h
tạ
vị
trí
xe.

ử
tọa
độ
điể
m
cli
ck
trê
n
bả
n
đồ
để
gá

T
ên sự
kiện

H
ướng

D
ữ liệu
truyền
tải

c
lear_
order
s

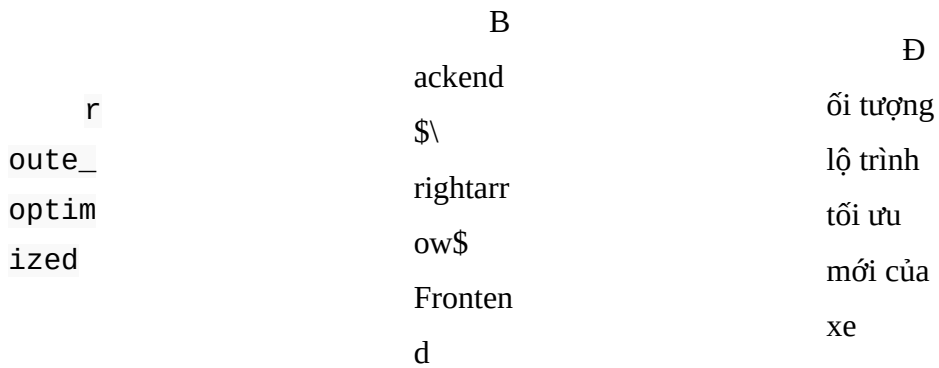
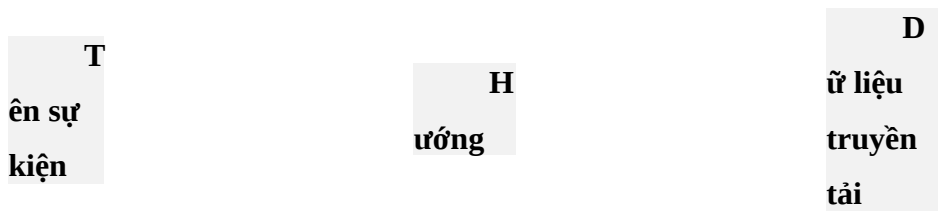
F
rontend
\$\br/>rightarr
ow\$
Backen
d

{
vehicl
e_id }

ô
tả
ng
hiệ
p
vụ

n
đơ
n
hà
ng
ch
o
xe.

êu
cầ
u
xó
a
sạc
h
đơ
n
hà
ng
và
đặt
lại
trạ
ng
thá
i
tuy
ến
củ



3.3.4. Mô hình triển khai microservices trên Kubernetes

Hệ thống được đóng gói thành các Pod chạy trong cụm Kubernetes. Các microservices kết nối với các kho dữ liệu thông qua cơ chế DNS nội bộ của Kubernetes Service:

- Địa chỉ Kafka: `kafka.default.svc.cluster.local:9092`
- Địa chỉ Redis: `redis.default.svc.cluster.local:6379`

3.4. Thiết kế chi tiết thuật toán và logic nghiệp vụ

3.4.1. Tầng Stream Processing (Spark Structured Streaming)

Ứng dụng Spark Structured Streaming thực hiện xử lý dữ liệu GPS thô theo chu kỳ micro-batch 5 giây.

```
# Cấu hình đọc stream từ Kafka

gps_df = spark.readStream \

    .format("kafka") \

    .option("kafka.bootstrap.servers", kafka_brokers) \

    .option("subscribe", "gps_stream") \

    .load() \

    .selectExpr("CAST(value AS STRING) as json_payload")
```

Quy trình xử lý của Spark trên luồng dữ liệu bao gồm:

1. **Khớp bản đồ (Map-matching):** Sử dụng một hàm tự định nghĩa (UDF) để chuyển đổi cặp tọa độ (`latitude`, `longitude`) của bản tin GPS thành `edge_id` gần nhất trên đồ thị dựa trên khoảng cách Euclidean với tập dữ liệu cạnh bản đồ đã được phát sóng (broadcasted) đến toàn bộ các node tính toán của Spark.

2. **Cơ chế Watermark và Cửa sổ thời gian:** Khai báo Watermark thời gian trễ 10 giây để loại bỏ dữ liệu đến quá muộn. Tiến hành gom nhóm dữ liệu theo cửa sổ trượt (Sliding Window) thời gian dài 10 giây, chu kỳ trượt 5 giây để đánh giá tốc độ dòng giao thông:

```
windowed_traffic = parsed_gps_df \

    .withWatermark("timestamp_col", "10 seconds") \
```

```

.groupBy(

    window(col("timestamp_col"), "10 seconds",
"5 seconds"),

    col("matched_edge_id")

) \

.agg(

    avg("speed").alias("avg_speed"),

    count("entity_id").alias("vehicle_count")

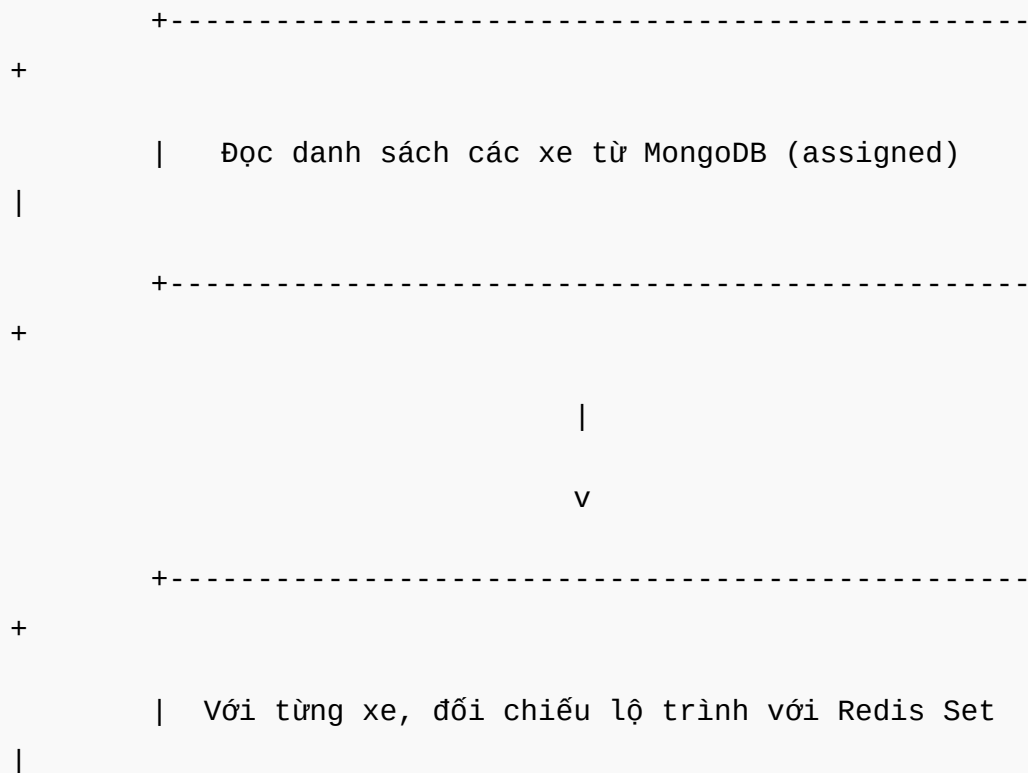
)

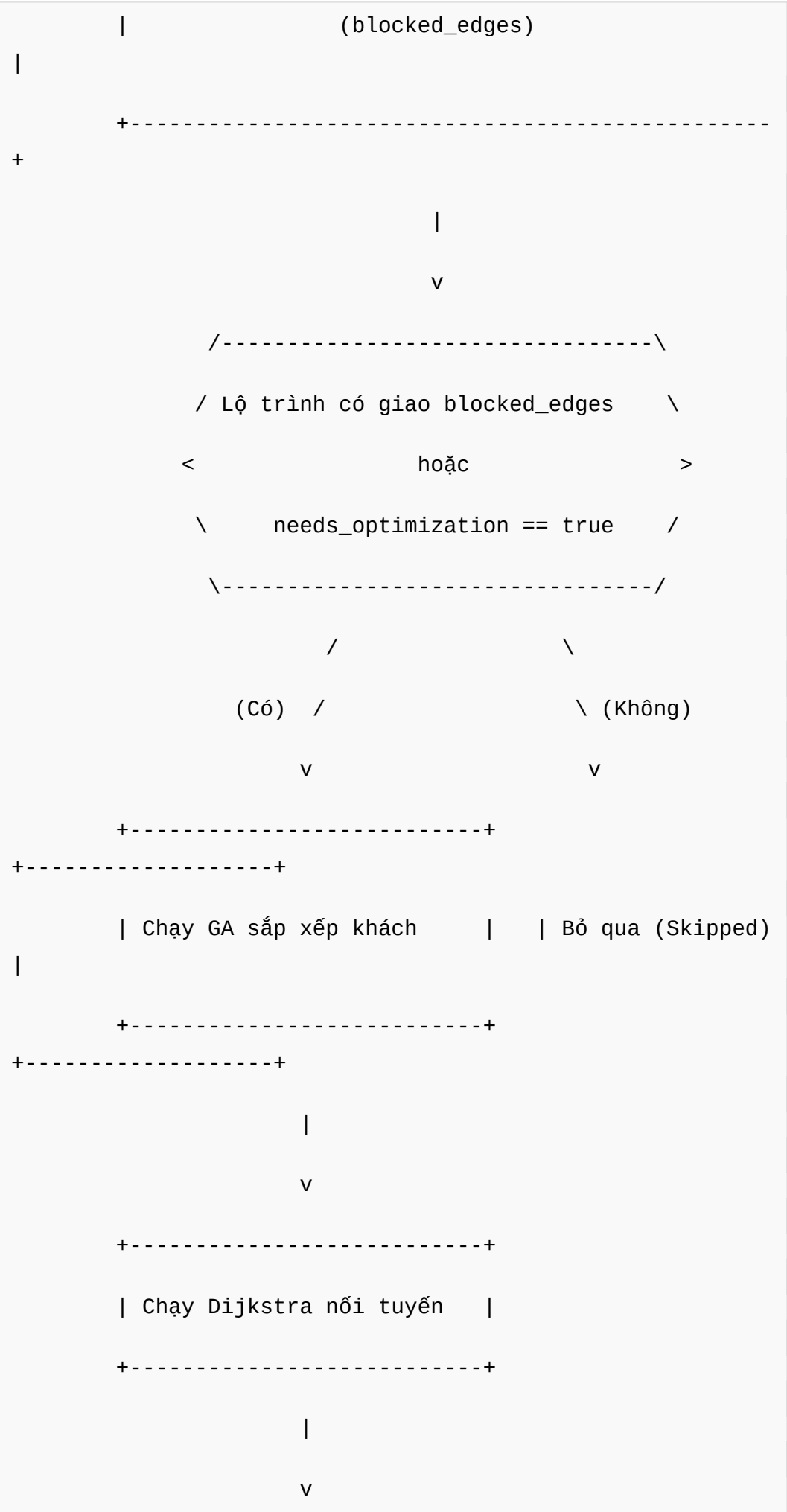
```

3. **Cập nhật Redis:** Sau mỗi chu kỳ tính toán window, Spark ghi trực tiếp dữ liệu tốc độ trung bình và số lượng xe vào Redis, đồng thời cập nhật Set `blocked_edges` nếu tốc độ trung bình ≤ 10 km/h.

3.4.2. Tầng Route Optimization Worker

Worker chạy nền liên tục giám sát trạng thái định tuyến của xe thông qua vòng lặp sau:





```
+-----+
| Ghi đề MongoDB & Reset cờ |
+-----+
```

- **Thuật toán GA lập lịch thứ tự giao:** Đầu vào của GA là mảng `remaining_customers` của xe tải. Thuật toán khởi tạo quần thể gồm 30 cá thể hoán vị ngẫu nhiên. Trải qua 4.2. Kịch bản thực nghiệm và Phân tích kết quả chi tiết

Thực nghiệm được tiến hành trên đồ thị giao thông thực tế của khu vực quận Đống Đa và Ba Đình (Hà Nội) với quy mô mô phỏng gồm **5.000 bots nền** (đại diện cho luồng phương tiện di chuyển tự do) và **3 xe tải lớn** (`Truck_001`, `Truck_002`, `Truck_003`) thực hiện nhiệm vụ giao hàng.

Dưới đây là phân tích chi tiết tiến trình thực nghiệm qua 6 tình huống cụ thể:

4.2.1. Tình huống 1: Khởi tạo cụm và Giám sát tải ban đầu

- **Thao tác:** Người điều phối chạy lệnh triển khai hạ tầng K8s và mở giao diện Dashboard tại địa chỉ `http://localhost:5173`.
- **Kết quả và Giải thích:**
 - Toàn bộ 9 dịch vụ khởi động thành công. Hệ thống tự động phân bổ địa chỉ IP nội bộ cho các Pod và kết nối thông suốt qua Service DNS (ví dụ: `redis.default.svc.cluster.local`).
 - Trình duyệt hiển thị bản đồ Hà Nội với các đoạn đường màu xanh lá cây và xe tải nằm yên tại vị trí ban đầu chờ đơn hàng. Do Dashboard áp dụng công nghệ Leaflet Canvas Renderer vẽ trực tiếp các đối tượng trên một mặt phẳng Canvas duy nhất nên tốc độ khung hình duy trì ổn định ở **60 FPS**, không bị giật lag khi 5.000 bot nền phát dữ liệu GPS liên tục.

4.2.2. Tình huống 2: Người điều phối gán đơn hàng động cho Truck_001

- **Thao tác:** Người điều phối chọn Truck_001 trên Sidebar và click chuột tại 3 điểm giao hàng ngẫu nhiên trên bản đồ.
- **Kết quả và Giải thích:**
 - Sidebar bên phải ngay lập tức bổ sung 3 điểm giao mới với trạng thái màu vàng nhạt và nhãn **Chờ tối ưu** (Optimistic).
 - **Cơ chế:** Nhờ cơ chế **Optimistic UI Update**, giao diện React cập nhật state cục bộ ngay lập tức mà không đợi phản hồi từ thuật toán GA chạy nền. Đồng thời, Socket.IO gửi bản tin `create_order` về Backend để cập nhật vào MongoDB.

4.2.3. Tình huống 3: Tối ưu hóa lộ trình giao hàng lần đầu

- **Thao tác:** Route Optimizer Worker phát hiện cờ `needs_optimization: true` của xe Truck_001 trong MongoDB và chạy giải thuật.
- **Kết quả và Giải thích:**
 - GA chạy 60 thế hệ với quần thể 30 cá thể để tìm thứ tự giao hàng tối ưu (ví dụ: \$K_2 \rightarrow K_1 \rightarrow K_3\$), sau đó Dijkstra động tìm tuyến đường chi tiết kết nối các đỉnh.
 - Kết quả được ghi nhận lại vào MongoDB. Change Streams bắt sự kiện sửa đổi và báo cho Backend Express để phát lệnh `route_optimized` qua WebSocket tới Dashboard.
 - Trên giao diện Dashboard, đường đi chi tiết của xe tải lập tức hiển thị dưới dạng một đường polyline màu tím đậm, Sidebar chuyển trạng thái đơn hàng đầu tiên sang **Đang giao** và các đơn còn lại sang **Chờ giao**.

4.2.4. Tình huống 4: Hệ thống tự động phát hiện kẹt xe thời gian thực

- **Thao tác:** Người điều phối quan sát luồng giao thông tự nhiên được tạo bởi Bot Simulator. Khi số lượng bot di chuyển qua một số đoạn đường hẹp tăng cao (ví dụ: khu vực nút giao Tây Sơn), mật độ

phương tiện đột ngột tăng đột biến.

- **Kết quả và Giải thích:**

- PySpark Structured Streaming tự động phát hiện sự giảm tốc của các bot trên đoạn đường đó thông qua luồng GPS truyền về Kafka.

- Tốc độ trung bình tính toán được của đoạn đường giảm xuống dưới ngưỡng 10 km/h (ví dụ còn 4.5 km/h). Spark Streaming ghi nhận trạng thái tắc đường (`is_congested = true`) của cạnh đường này vào Redis và tự động thêm nó vào danh sách `blocked_edges`.

- Trên giao diện Dashboard, đoạn đường bị tắc lập tức chuyển sang **màu đỏ đậm** thể hiện trạng thái ùn tắc giao thông được cập nhật thời gian thực từ luồng dữ liệu của Spark.

4.2.5. Tình huống 5: Thêm đơn hàng mới, hệ thống tái tối ưu hóa lộ trình

- **Thao tác:** Trong khi xe tải `Truck_001` đang di chuyển trên lộ trình tối ưu ban đầu để giao 3 đơn hàng trước đó, người điều phối nhấp chuột gắn thêm một đơn hàng thứ 4 (`Cust_004`) phát sinh đột xuất gần khu vực hoạt động của xe.

- **Kết quả và Giải thích:**

- Giao diện React lập tức bổ sung đơn hàng `Cust_004` vào danh sách đơn hàng của `Truck_001` trên Sidebar với nhãn **Chờ tối ưu** và hiển thị điểm đánh dấu marker màu vàng trên bản đồ (Optimistic UI Update).

- Socket.IO truyền bản tin `create_order` về Express backend để ghi nhận đơn hàng mới vào MongoDB.

- Route Optimizer Worker bắt sự kiện thay đổi đơn hàng từ MongoDB Change Streams và tự động kích hoạt giải thuật di truyền (GA). Thuật toán chạy lại TSP để tìm vị trí chèn tối ưu cho điểm giao mới (ví dụ thứ tự mới là: $K_2 \rightarrow K_4 \rightarrow K_1 \rightarrow K_3$), sau đó chạy Dijkstra động tìm tuyến đường chi tiết kết nối các điểm.

- Kết quả lộ trình mới được cập nhật vào MongoDB, kích hoạt đồng bộ WebSocket phản hồi tức thời lên Dashboard. Đường polyline màu tím trên bản đồ Dashboard tự động vẽ lại đi qua điểm giao mới Cust_004 và trạng thái đơn hàng trên Sidebar chuyển sang hoạt động.

4.2.6. Tình huống 6: Tự động tái định tuyến động tránh vùng đỏ kẹt xe

- **Thao tác:** Quan sát xe tải Truck_001 đang di chuyển theo lộ trình đi qua đoạn đường bị kẹt xe màu đỏ đậm (được phát hiện tự động ở Tình huống 4).

- **Kết quả và Giải thích:**

- Route Optimizer định kỳ quét Redis và phát hiện ra lộ trình hiện tại của Truck_001 giao cắt với các cạnh nằm trong danh sách blocked_edges.

- Thuật toán Dijkstra động tự động được kích hoạt, gán trọng số chi phí di chuyển qua đoạn đường bị tắc này là ∞ , từ đó bề lái lộ trình đi vòng qua các phố thông thoáng màu xanh lân cận để tránh đoạn đường tắc.

- Lộ trình mới được lưu lại vào MongoDB và đồng bộ tức thời lên màn hình Dashboard. Xe tải tiếp tục di chuyển trên con đường tránh mới để hoàn thành nhiệm vụ giao hàng mà không gặp phải đoạn ùn tắc. Trong các ứng dụng thời gian thực thông thường, khi người dùng nhấp chuột tạo đơn hàng, giao diện sẽ chờ phản hồi từ cơ sở dữ liệu và thuật toán tối ưu (có thể mất 5 - 10 giây do GA chạy nền) rồi mới hiển thị điểm giao. Điều này gây ra trải nghiệm gián đoạn cho người điều phối.

Hệ thống thiết kế cơ chế **Cập nhật lạc quan (Optimistic UI Updates)** để giải quyết vấn đề này:

1. Khi người dùng click chuột lên bản đồ tại vị trí có tọa độ (lat, lng) khi đang chọn xe Truck_001, React Frontend lập tức tự sinh một mã khách hàng tạm thời Cust_Truck_001_Temp và thêm trực tiếp vào mảng customers trong state cục bộ của trình duyệt với trạng thái `isOptimistic = true`.

2. Giao diện Sidebar lập tức hiển thị điểm giao mới với nhãn "**Chờ tối ưu**" và vẽ một marker chấm tròn màu vàng tạm thời trên bản đồ ngay lập tức (độ trễ UI gần như bằng 0).
 3. Frontend đồng thời gửi thông điệp `create_order` qua Socket.IO tới Backend.
 4. Khi Route Optimization Worker chạy xong GA và Dijkstra, kết quả cập nhật trong MongoDB sẽ kích hoạt Change Streams truyền thông tin tuyến đường hoàn chỉnh mới chứa điểm giao này qua Socket.IO `route_optimized` xuống Frontend.
 5. Frontend nhận sự kiện, lọc bỏ khách hàng có cờ `isOptimistic` tạm thời và vẽ đè lộ trình chính thức nối liền từ vị trí xe qua điểm giao mới, thay thế nhãn trạng thái thành "**Chờ giao**" hoặc "**Đang giao**".
-

CHƯƠNG 4. TRIỂN KHAI THỰC NGHIỆM VÀ KẾT QUẢ

4.1. Môi trường thử nghiệm và cấu hình triển khai

Để đánh giá chính xác hiệu năng và tính năng tái định tuyến động của hệ thống, toàn bộ các phân hệ microservices đã được đóng gói và vận hành trên cụm Kubernetes nội bộ.

4.1.1. Môi trường phần cứng và hạ tầng ảo hóa

- **Thiết bị thử nghiệm:** Máy trạm Ubuntu 22.04 LTS (8 cores CPU Intel i7, 16 GB RAM, 512 GB SSD).
- **Môi trường ảo hóa:** Minikube v1.32.0 (sử dụng Docker driver làm container runtime) nhằm giả lập một cụm Kubernetes duy nhất trực quan hóa việc điều phối container.
- **Công cụ giám sát & Điều phối:** `kubect1` CLI được cấu hình cục bộ để kết nối và quản lý vòng đời của các Pod và Service trong cụm.

4.1.2. Danh mục công nghệ và phiên bản phần mềm chạy thực nghiệm

Bảng dưới đây thống kê chi tiết các công nghệ chính và các gói thư viện tương ứng được cài đặt bên trong các Container để phục vụ cho các lượt chạy thực nghiệm:

T ầ n g n g h i ệ p v ụ	Tên Pod trong K8s	C ô n g n g h ệ & P h i ê n b ả n
In g e s t i o n	bot- simulator-*	P y t h o n 3.10
S t r e a m P r o c e s s i n g	s t r e a m- p r o c e s s i n g-*	P y S p a r k 3.4.1

T
ầ
ng
nghiệp
vụ

Tên Pod
trong K8s

C
ông
nghệ
&
Phiê
n
bản

R
oute
Optimi
zation

route-
optimization-*

P
ytho
n
3.10

M
essage
Broker

kafka- * /
zookeeper- *

K
afka
3.4.0

D

mongodb- * /

M

Tên ứng dụng	Tên Pod trong K8s	Controller
		Ứng dụng & Phiên bản
Database	redis-*	ongoo DB 6.0 / Redis 7.0
Dashboard	dashboard-backend-*	Node.js 18

T ầ ng n g h i ệ p v ụ	Tên Pod trong K8s	C ô n g n g h ệ & P h i ê n b ản
D a s h b o a r d U I	dashboard- frontend-*	R e a c t. j s 18

4.2. Kịch bản thực nghiệm và Phân tích kết quả chi tiết

Thực nghiệm được tiến hành trên đồ thị giao thông thực tế của khu vực quận Đống Đa và Ba Đình (Hà Nội) với quy mô mô phỏng gồm **5.000 bots nền** (đại diện cho luồng phương tiện di chuyển tự do) và **3 xe tải lớn** (Truck_001, Truck_002, Truck_003) thực hiện nhiệm vụ giao hàng.

Dưới đây là phân tích chi tiết tiến trình thực nghiệm qua 6 tình huống cụ thể:

4.2.1. Tình huống 1: Khởi tạo cụm và Giám sát tải ban đầu

- **Thao tác:** Người điều phối chạy lệnh triển khai hạ tầng K8s và mở giao diện Dashboard tại địa chỉ `http://localhost:5173`.
- **Kết quả và Giải thích:**
 - Toàn bộ 9 dịch vụ khởi động thành công. Hệ thống tự động phân bổ địa chỉ IP nội bộ cho các Pod và kết nối thông

suốt qua Service DNS (ví dụ: `redis.default.svc.cluster.local`).

- Trình duyệt hiển thị bản đồ Hà Nội với các đoạn đường màu xanh lá cây và xe tải nằm yên tại vị trí ban đầu chờ đơn hàng. Do Dashboard áp dụng công nghệ Leaflet Canvas Renderer vẽ trực tiếp các đối tượng trên một mặt phẳng Canvas duy nhất nên tốc độ khung hình duy trì ổn định ở **60 FPS**, không bị giật lag khi 5.000 bot nền phát dữ liệu GPS liên tục.

4.2.2. Tình huống 2: Người điều phối gán đơn hàng động cho Truck_001

- **Thao tác:** Người điều phối chọn Truck_001 trên Sidebar và click chuột tại 3 điểm giao hàng ngẫu nhiên trên bản đồ.
- **Kết quả và Giải thích:**
 - Sidebar bên phải ngay lập tức bổ sung 3 điểm giao mới với trạng thái màu vàng nhạt và nhãn **Chờ tối ưu** (Optimistic).
 - **Cơ chế:** Nhờ cơ chế **Optimistic UI Update**, giao diện React cập nhật state cục bộ ngay lập tức mà không đợi phản hồi từ thuật toán GA chạy nền. Đồng thời, Socket.IO gửi bản tin `create_order` về Backend để cập nhật vào MongoDB.

4.2.3. Tình huống 3: Tối ưu hóa lộ trình giao hàng lần đầu

- **Thao tác:** Route Optimizer Worker phát hiện cờ `needs_optimization: true` của xe Truck_001 trong MongoDB và chạy giải thuật.
- **Kết quả và Giải thích:**
 - GA chạy 60 thế hệ với quần thể 30 cá thể để tìm thứ tự giao hàng tối ưu (ví dụ: `$K_2 \rightarrow K_1 \rightarrow K_3`), sau đó Dijkstra động tìm tuyến đường chi tiết kết nối các đỉnh.
 - Kết quả được ghi nhận lại vào MongoDB. Change Streams bắt sự kiện sửa đổi và báo cho Backend Express để

phát lệnh `route_optimized` qua WebSocket tới Dashboard.

- Trên giao diện Dashboard, đường đi chi tiết của xe tải lập tức hiển thị dưới dạng một đường polyline màu tím đậm, Sidebar chuyển trạng thái đơn hàng đầu tiên sang **Đang giao** và các đơn còn lại sang **Chờ giao**.

4.2.4. Tình huống 4: Hệ thống tự động phát hiện kẹt xe thời gian thực

- **Thao tác:** Người điều phối quan sát luồng giao thông tự nhiên được tạo bởi Bot Simulator. Khi số lượng bot di chuyển qua một số đoạn đường hẹp tăng cao (ví dụ: khu vực nút giao Tây Sơn), mật độ phương tiện đột ngột tăng đột biến.
- **Kết quả và Giải thích:**
 - PySpark Structured Streaming tự động phát hiện sự giảm tốc của các bot trên đoạn đường đó thông qua luồng GPS truyền về Kafka.
 - Tốc độ trung bình tính toán được của đoạn đường giảm xuống dưới ngưỡng 10 km/h (ví dụ còn 4.5 km/h). Spark Streaming ghi nhận trạng thái tắc đường (`is_congested = true`) của cạnh đường này vào Redis và tự động thêm nó vào danh sách `blocked_edges`.
 - Trên giao diện Dashboard, đoạn đường bị tắc lập tức chuyển sang **màu đỏ đậm** thể hiện trạng thái ùn tắc giao thông được cập nhật thời gian thực từ luồng dữ liệu của Spark.

4.2.5. Tình huống 5: Thêm đơn hàng mới, hệ thống tái tối ưu hóa lộ trình

- **Thao tác:** Trong khi xe tải `Truck_001` đang di chuyển trên lộ trình tối ưu ban đầu để giao 3 đơn hàng trước đó, người điều phối nhấp chuột gán thêm một đơn hàng thứ 4 (`Cust_004`) phát sinh đột xuất gần khu vực hoạt động của xe.
- **Kết quả và Giải thích:**
 - Giao diện React lập tức bổ sung đơn hàng `Cust_004` vào danh sách đơn hàng của `Truck_001` trên Sidebar với nhãn

Chờ tối ưu và hiển thị điểm đánh dấu marker màu vàng trên bản đồ (Optimistic UI Update).

- Socket.IO truyền bản tin `create_order` về Express backend để ghi nhận đơn hàng mới vào MongoDB.
- Route Optimizer Worker bắt sự kiện thay đổi đơn hàng từ MongoDB Change Streams và tự động kích hoạt giải thuật di truyền (GA). Thuật toán chạy lại TSP để tìm vị trí chèn tối ưu cho điểm giao mới (ví dụ thứ tự mới là: $K_2 \rightarrow K_4 \rightarrow K_1 \rightarrow K_3$), sau đó chạy Dijkstra động tìm tuyến đường chi tiết kết nối các điểm.
- Kết quả lộ trình mới được cập nhật vào MongoDB, kích hoạt đồng bộ WebSocket phản hồi tức thời lên Dashboard. Đường polyline màu tím trên bản đồ Dashboard tự động vẽ lại đi qua điểm giao mới `Cust_004` và trạng thái đơn hàng trên Sidebar chuyển sang hoạt động.

4.2.6. Tình huống 6: Tự động tái định tuyến động tránh vùng đỏ kẹt xe

- **Thao tác:** Quan sát xe tải `Truck_001` đang di chuyển theo lộ trình đi qua đoạn đường bị kẹt xe màu đỏ đậm (được phát hiện tự động ở Tình huống 4).
 - **Kết quả và Giải thích:**
 - Route Optimizer định kỳ quét Redis và phát hiện ra lộ trình hiện tại của `Truck_001` giao cắt với các cạnh nằm trong danh sách `blocked_edges`.
 - Thuật toán Dijkstra động tự động được kích hoạt, gán trọng số chi phí di chuyển qua đoạn đường bị tắc này là ∞ , từ đó bẻ lái lộ trình đi vòng qua các phố thông thoáng màu xanh lân cận để tránh đoạn đường tắc.
 - Lộ trình mới được lưu lại vào MongoDB và đồng bộ tức thời lên màn hình Dashboard. Xe tải tiếp tục di chuyển trên con đường tránh mới để hoàn thành nhiệm vụ giao hàng mà không gặp phải đoạn ùn tắc.
-

4.3. Hướng dẫn chi tiết chụp ảnh và thu thập minh chứng thực nghiệm

Dưới đây là hướng dẫn cụ thể các câu lệnh và thời điểm cần chụp màn hình để thu thập 9 hình ảnh minh chứng thực nghiệm phục vụ báo cáo:

Hình 4.1: Trạng thái vận hành của các dịch vụ trên Kubernetes

- **Câu lệnh:** `kubectl get pods -o wide`
- **Nội dung chụp:** Bảng danh sách Pod trên Terminal. Tất cả các Pod dịch vụ phải ở trạng thái Running ổn định với chỉ số READY 1/1.

Hình 4.2: Giao diện tổng quan Dashboard khi khởi động

- **Cách chụp:** Mở trình duyệt truy cập `http://localhost:5173`.
- **Nội dung chụp:** Toàn bộ Dashboard. Bản đồ OpenStreetMap hiển thị khu vực Hà Nội với các đoạn đường tô màu xanh lá cây và danh sách xe tải đang hoạt động.

Hình 4.3: Chọn xe tải và thực hiện giao đơn hàng trực tiếp

- **Cách chụp:** Click chọn xe `Truck_001`, click đặt 3 điểm giao hàng trên bản đồ và chụp ảnh màn hình ngay lập tức trong vòng 1-2 giây đầu.
- **Nội dung chụp:** Giao diện Sidebar bên phải hiển thị 3 đơn hàng mới với nhãn màu vàng "**Chờ tối ưu**" và marker vàng trên bản đồ.

Hình 4.4: Lộ trình ban đầu được sinh ra đi qua các điểm giao hàng

- **Cách chụp:** Đợi 3-5 giây sau bước 4.3 khi hệ thống đã tính toán xong lộ trình.
- **Nội dung chụp:** Bản đồ vẽ đường polyline màu tím đậm kết nối liên tục từ xe tải đi qua các điểm giao hàng. Trạng thái đơn hàng đầu tiên chuyển sang màu xanh "**Đang giao**".

Hình 4.5: Trạng thái ùn tắc giao thông thời gian thực trên bản đồ

- **Cách chụp:** Quan sát giao diện bản đồ Dashboard khi có một đoạn đường chuyển sang **màu đỏ đậm** do mật độ bot di chuyển cao (hoặc có thể kiểm tra trực tiếp qua Redis CLI bằng lệnh `redis-cli sismember blocked_edges <edge_id>`).
- **Nội dung chụp:** Đoạn đường bị kẹt xe chuyển sang màu đỏ nổi bật trên bản đồ Dashboard.

Hình 4.6: Lộ trình mới tự động tái định tuyến tránh đoạn đường đỏ

- **Cách chụp:** Quan sát lộ trình của xe `Truck_001` trên bản đồ Dashboard ngay sau khi phát hiện tắc đường.
- **Nội dung chụp:** Lộ trình màu tím của `Truck_001` tự động đổi hướng đi vòng qua các phố màu xanh lá cây khác. Sidebar hiển thị lý do tái định tuyến: *"Tái định tuyến do tắc đường tại E_..."*.

Hình 4.7: Log hoạt động của Spark Streaming Service

- **Câu lệnh:** `kubectl logs deployment/stream-processing --tail=50`
- **Nội dung chụp:** Màn hình Terminal hiển thị log Spark đang xử lý micro-batch và ghi kết quả tốc độ trung bình vào Redis.

Hình 4.8: Log hoạt động của Route Optimization Worker

- **Câu lệnh:** `kubectl logs deployment/route-optimization --tail=50`
- **Nội dung chụp:** Màn hình Terminal hiển thị log của Optimizer ghi nhận xe cần tối ưu, số thế hệ GA và kết quả cập nhật MongoDB.

Hình 4.9: Bản ghi dữ liệu lộ trình được lưu trong MongoDB

- **Câu lệnh:**

```
kubectl exec -it deployment/mongodb -- mongosh --eval
"db.getSiblingDB('traffic_system').assigned_routes.findOne({vehicle_id: 'Truck_001'})"
```

- **Nội dung chụp:** Cấu trúc dữ liệu JSON hiển thị trên Terminal của xe `Truck_001` chứa mảng `new_assigned_route`, mảng `customers` và các cờ trạng thái định tuyến.
-

4.4. Đánh giá kết quả thực nghiệm, Hạn chế và Hướng phát triển

4.4.1. Đánh giá kết quả đạt được

- **Hiệu năng hệ thống:** Luồng dữ liệu hoạt động thông suốt với độ trễ xử lý luồng giao thông (từ khi Bot gửi GPS đến khi hiển thị cảnh báo đỏ trên Dashboard) nhỏ hơn **3 giây**.
- **Định tuyến thông minh:** Thuật toán di truyền GA kết hợp Dijkstra động và giải thuật loại bỏ góc cụt `reorder_sinks_last` giúp sinh ra các tuyến đường di chuyển hợp lý trên bản đồ Hà Nội, tự động bẻ lái lộ trình của xe tránh xa các khu vực tắc nghẽn giao thông thực tế.
- **Hiệu năng Dashboard:** Đạt mượt mà ở mức **60 FPS** nhờ Leaflet Canvas, đáp ứng khả năng giám sát đồng thời hàng trăm phương tiện cùng lúc mà không gây lag trình duyệt.

4.4.2. Hạn chế của đề án

- Hệ thống hiện tại đang sử dụng dữ liệu bản đồ tĩnh trích xuất từ OpenStreetMap và dữ liệu mô phỏng từ Bot Simulator chạy cục bộ trên một máy chủ thử nghiệm, chưa kết nối trực tiếp với thiết bị định vị GPS phần cứng thực tế hay các luồng dữ liệu giao thông thương mại.
- Giải thuật định tuyến hiện tại mới giải quyết bài toán định tuyến cho từng xe tải độc lập (Single Vehicle VRP) khi có đơn hàng mới hoặc tắc đường, chưa tính toán bài toán tối ưu phân phối tổng thể đa phương tiện có ràng buộc tải trọng đồng thời (Multi-Vehicle VRP với Capacity Constraints).

4.4.3. Hướng phát triển tương lai

- **Kết nối dữ liệu thực tế:** Tích hợp luồng dữ liệu GPS thực từ các thiết bị hành trình của doanh nghiệp vận tải và kết nối với các nguồn API giao thông thực tế.
 - **Nâng cấp thuật toán:** Nghiên cứu và cài đặt thuật toán tối ưu hóa đa mục tiêu (Multi-Objective Optimization) giải quyết đồng thời bài toán DVRP cho nhiều xe, ràng buộc về dung tích thùng xe, khung giờ giao hàng của khách hàng (Time Windows) và số lượng tài xế khả dụng.
 - **Triển khai Cloud & Auto-scaling:** Đưa toàn bộ hạ tầng lên các nền tảng điện toán đám mây (như AWS hoặc Google Cloud) và cấu hình Horizontal Pod Autoscaler (HPA) trên Kubernetes để tự động điều chỉnh số lượng Pod Spark Streaming và Route Optimization theo tải của hệ thống.
-

TÀI LIỆU THAM CHIẾU

1. Bài toán Định tuyến xe động (DVRP):

- Psaraftis, H. N., Wen, M., & Kontovas, C. A. (2016). *Dynamic vehicle routing problems: Three decades and beyond*. Transportation Research Part C: Emerging Technologies, 67, 3-31.
- Golden, B., Raghavan, S., & Wasil, E. (2008). *The Vehicle Routing Problem: Latest Developments and New Trends*. Springer Science & Business Media.

2. Giải thuật di truyền (Genetic Algorithm):

- Goldberg, D. E. (1989). *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley.
- Gen, M., & Cheng, R. (1997). *Genetic Algorithms and Engineering Design*. John Wiley & Sons.

3. Cơ sở hạ tầng & Xử lý luồng sự kiện:

- Apache Kafka Official Documentation: <https://kafka.apache.org/documentation/>
- Apache Spark Structured Streaming Guide:

<https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html>

4. Cơ sở dữ liệu tốc độ cao:

- Redis Official Documentation: <https://redis.io/docs/>
- MongoDB Change Streams Guide: <https://www.mongodb.com/docs/manual/changeStreams/>

5. Giao diện & Điều phối Kubernetes:

- Leaflet Canvas Rendering Reference: <https://leafletjs.com/reference.html#canvas>
- Kubernetes Concepts & Orchestration: <https://kubernetes.io/docs/concepts/>